

CARTMAN AND THE GOLDEN CUBE

Hecho por Yura Hernández y Juan José Bolívar

Pontificia Universidad Javeriana Cali

Proyecto Final Arboles y Grafos

Profesor: Carlos Ramírez

Descripción general del problema:

El problema consiste en un cubo que rueda por una cuadrícula y que al caer sobre cada celda intercambia el oro que tiene en su cara inferior con el oro que pueda haber en esa celda. En el mapa hay exactamente seis pedazos de oro distribuidos y el objetivo es encontrar la ruta más económica para que el cubo los recoja todos simultáneamente, es decir, que los seis queden dentro del cubo al mismo tiempo. La dificultad radica en que, como el cubo es tridimensional y va rodando, se debe llevar un registro bastante fino sobre de qué cara está apuntando hacia abajo en cada momento, esto principalmente porque cada cara es independiente y puede o no contener oro. Además, cada movimiento tiene un costo que varía dependiendo de que, si en ese paso se recogió un nuevo pedazo de oro o no, lo que convierte el problema en una búsqueda del camino óptimo considerando simultáneamente la posición del cubo, su orientación y el estado de recolección del oro. Por esto es que nosotros al darnos cuenta de búsqueda de caminos óptimos, pensamos principalmente en un Dijkstra. (La forma en como llegamos a esa conclusión está especificada mejor en la bitácora)

Especificación del problema:

Entrada: $A, B \in \mathbb{N}$, una matriz $M[0..r][0..c] \mid M[i,j] \in \sum$ donde $\sum \in \{“.”, “S”, “G”\} \mid \{(i,j): M[i,j] = “G”\} = 6 \mid \{(i,j): M[i,j] = “S”\} = 1$

Salida: Un entero $X \in \mathbb{N} \cup \{-1\}$ donde $x =$ El costo mínimo requerido para que las 6 caras tengan oro, que se obtiene de sumar los pesos b cuando la cara inferior esta vacia y $M(r,c) =$

“G” y costo a en cualquier otro caso, si no es posible obtener los 6 oros entonces $x = -1$.

Definición de casos de prueba:

Vamos a decir que los estados se ven de la siguiente forma

$([0,0,0,0,0,0], (x,y), 0, 1, [(x,y), (x,y), (x,y), (x,y), (x,y), (x,y)])$

Donde la primera lista representa si cada cara tiene o no oro, la segunda la posición del dado, la tercera el costo acumulado, la cuarta es en que cara estamos (La parte inferior) siendo de 1 a 6 debido a que tenemos 6 caras y la última representa la posición de cada oro. La representación en los casos de prueba es una traza de ejecución para ilustrar el recorrido, no el estado formal.

Tenemos la siguiente entrada:

4 4 1 2
S . . .
G . G .
G G . .
G . . G

El estado inicial sería:

(
[0,0,0,0,0,0],
(0,0),
0,
[(1,0), (1,2), (2,0), (2,1), (3,0), (3,3)]
)

Abajo:

(
[1,0,0,0,0,0],
(1,0),
2,

[(1,2), (2,0), (2,1), (3,0), (3,3)]
)

Derecha:

(
[0,0,1,0,0,0],
(1,1),
3,
[(1,2), (2,0), (2,1), (3,0), (3,3)]
)

Derecha:

(
[1,0,0,0,0,1],
(1,2),
5,
[(2,0), (2,1), (3,0), (3,3)]
)

Derecha:

(
[0,0,1,0,1,0],
(1,3),
6,
[(2,0), (2,1), (3,0), (3,3)]
)

Abajo:

(
[0,0,1,0,1,0],
(2,3),
7,
[(2,0), (2,1), (3,0), (3,3)]
)

Izquierda:

(
[0,0,0,0,0,1],
(2,2),
8,
[(2,0), (2,1), (2,2), (3,0), (3,3)]
)

Derecha:

(

[0,0,0,0,1,0],
(2,3),
9,
[(2,0), (2,1), (2,2), (3,0), (3,3)]
)

Abajo:

(
[1,0,0,0,1,0],
(3,3),
11,
[(2,0), (2,1), (2,2), (3,0)]
)

Izquierda:

(
[0,0,0,0,1,1],
(3,2),
12,
[(2,0), (2,1), (2,2), (3,0)]
)

Arriba:

(
[1,1,0,0,1,0],
(2,2),
14,
[(2,0), (2,1), (3,0)]
)

Izquierda:

(
[1,1,0,0,1,1],
(2,1),
16,
[(2,0), (3,0)]
)

Izquierda:

(
[1,1,1,0,1,1],
(2,0),
18,
[(3,0)]
)

Abajo:

```
(  
[1,1,1,1,1,1],  
(3,0),  
20,  
[]  
)
```

Esto termina el caso dándonos como salida:

Screw you guys, I got all the gold for 20 cost!

Caso #2)

```
4 4 1 2  
S . . .  
G . . .  
G G G .  
. . G G
```

El estado inicial sería el siguiente:

```
(  
[0,0,0,0,0,0],  
(0,0),  
0,  
[(1,0),(2,0),(2,1),(2,2),(3,2),(3,3)]  
)
```

Abajo:

```
(  
[1,0,0,0,0,0],  
(1,0),  
2,  
[(2,0),(2,1),(2,2),(3,2),(3,3)]  
)
```

Abajo:

```
(  
[1,1,0,0,0,0],  
(2,0),  
4,  
[(2,1),(2,2),(3,2),(3,3)]  
)
```

Derecha:

```
(  
[1,1,1,0,0,0],  
(2,1),
```

```
6,  
[(2,2),(3,2),(3,3)]  
)
```

Derecha:

```
(  
[1,1,1,0,0,1],  
(2,2),  
8,  
[(3,2),(3,3)]  
)
```

Abajo:

```
(  
[1,1,1,1,0,1],  
(3,2),  
10,  
[(3,3)]  
)
```

Derecha:

```
(  
[1,1,1,1,1,1],  
(3,3),  
12,  
[]  
)
```

Aquí termina el caso y se tiene la siguiente salida:

Screw you guys, I got all the gold for 12 cost!

Caso #3)

Tenemos la siguiente entrada:

5 5 2 4

.. G ..

.. G ..

S G G G .

G

.

El estado inicial sería:

(
[0,0,0,0,0,0],
(2,0),
0,
[(0,2), (1,2), (2,1), (2,2), (2,3), (3,0)]
)

Abajo:

(
[1,0,0,0,0,0],
(3,0),
4,
[(0,2), (1,2), (2,1), (2,2), (2,3)]
)

Arriba:

(
[0,0,0,1,0,0],
(2,0),
6,
[(0,2), (1,2), (2,1), (2,2), (2,3)]
)

Derecha:

(
[1,0,0,1,0,0],
(2,1),
10,
[(0,2), (1,2), (2,2), (2,3)]
)

Derecha:

(
[1,0,1,1,0,0],
(2,2),
14,
[(0,2), (1,2), (2,3)]
)

)

Derecha:

(
[1,0,1,1,0,1],
(2,3),
18,
[(0,2), (1,2)]
)

Izquierda:

(
[0,0,1,1,1,0],
(2,2),
20,
[(0,2), (1,2), (2,2)]
)

Arriba:

(
[1,0,1,0,1,1],
(1,2),
24,
[(0,2), (2,2)]
)

Abajo:

(
[1,1,1,1,1,0],
(2,2),
28,
[(0,2)]
)

Arriba:

(
[0,0,1,1,1,1],
(1,2),
30,
[(0,2), (1,2)]
)

Arriba:

(
[1,1,1,0,1,1],
)

```
(0,2),
34,
[(1,2)]
)
```

Abajo:

```
(
[1,1,1,1,1,1],
(1,2),
38,
[]
)
```

Esto termina el caso dándonos como salida:

Screw you guys, I got all the gold for 38 cost!

Solucion al problema:

Para este problema al ser un grafo de estados ponderado decidimos hacer uso del algoritmo de búsqueda de Dijkstra. Los estados a la hora de representarlos en el programa decidimos que usaríamos una representación diferente ya que la complejidad aumenta a medida que añadimos mas elementos al estado, así que de esta forma decidimos lo siguiente:

Un entero para la fila

Un entero para la columna

Un entero para la representación del dado

Esto lo compactamos en un solo entero tomando en cuenta lo siguiente:

Como el máximo de la grilla es de 8x8 entonces lo máximo que ira tanto la fila y la columna es 7 de esta forma podemos en 3 bits representar estos valores. Por otro lado, como las caras siempre son 6 y queremos saber si tiene oro o no entonces podemos usar los 6 bits siguientes a la fila y columna de este entero.

Para las posiciones del oro optamos por representarlo en un entero también, como la grilla máxima es de 8x8 entonces tenemos 64 posiciones, las cuales con un uint64 podemos representarlo, de esta forma donde estén los oros ponemos un 1 y de resto 0.

Todo esto lo podemos poner en una priority queue y un unordered map para continuar con el algoritmo de Dijkstra.

Caso significativo:

5 5 5 10

.S..G

....G

..G..

G...G

...G.

El estado inicial sería:

```
(
[0,0,0,0,0,0],
(0,1),
0,
[(0,4), (1,4), (2,2), (3,0), (3,4), (4,3)]
)
```

Izquierda:

```
(
[0,0,0,0,0,0],
(0,0),
5,
[(0,4), (1,4), (2,2), (3,0), (3,4), (4,3)]
)
```

Abajo:

```
(
[0,0,0,0,0,0],
(1,0),
10,
[(0,4), (1,4), (2,2), (3,0), (3,4), (4,3)]
)
```

Abajo:

```
(
[0,0,0,0,0,0],
(2,0),
15,
[(0,4), (1,4), (2,2), (3,0), (3,4), (4,3)]
)
```

Abajo:

(
[1,0,0,0,0,0],
(3,0),
25,
[(0,4), (1,4), (2,2), (3,4), (4,3)]
)

Derecha:

(
[0,0,1,0,0,0],
(3,1),
30,
[(0,4), (1,4), (2,2), (3,4), (4,3)]
)

Derecha:

(
[0,0,0,0,0,1],
(3,2),
35,
[(0,4), (1,4), (2,2), (3,4), (4,3)]
)

Derecha:

(
[0,0,0,0,1,0],
(3,3),
40,
[(0,4), (1,4), (2,2), (3,4), (4,3)]
)

Arriba:

(
[0,0,0,0,1,0],
(2,3),
45,
[(0,4), (1,4), (2,2), (3,4), (4,3)]
)

Derecha:

(
[0,0,0,0,0,0],
(2,4),
50,
[(0,4), (1,4), (2,2), (2,4), (3,4), (4,3)]
)

Izquierda:

(
[0,0,0,0,0,0],
(2,3),
55,
[(0,4), (1,4), (2,2), (2,4), (3,4), (4,3)]
)

Izquierda:

(
[1,0,0,0,0,0],
(2,2),
65,
[(0,4), (1,4), (2,4), (3,4), (4,3)]
)

Abajo:

(
[0,1,0,0,0,0],
(3,2),
70,
[(0,4), (1,4), (2,4), (3,4), (4,3)]
)

Derecha:

(
[0,1,0,0,0,0],
(3,3),
75,
[(0,4), (1,4), (2,4), (3,4), (4,3)]
)

Abajo:

(
[1,0,0,0,0,1],
(4,3),
85,
[(0,4), (1,4), (2,4), (3,4)]
)

Derecha:

(
[0,0,1,0,1,0],
(4,4),

90,
[(0,4), (1,4), (2,4), (3,4)]
)

Arriba:

(
[1,0,1,0,1,0],
(3,4),
100,
[(0,4), (1,4), (2,4)]
)

Arriba:

(
[1,0,1,1,1,0],
(2,4),
110,
[(0,4), (1,4)]
)

Arriba:

(
[1,0,1,1,1,1],
(1,4),
120,
[(0,4)]
)

Arriba:

(
[1,1,1,1,1,1],
(0,4),
130,
[]
)

Esto termina el caso dándonos como salida:

Screw you guys, I got all the gold for 130 cost!

Bitacora del Proyecto final

Miércoles 29 de abril de 2026: El profesor Carlos Ramírez asigna el proyecto.

Jueves 30 de abril de 2026: Yura y Juan José deciden realizar el proyecto de manera conjunta.

Fin de semana de los días 1-3 de mayo: Antes de iniciar el proyecto decidimos la estrategia en la que vamos a afrontar este, en primera instancia pensamos en que Juan José afrontase principalmente la programación y yura la descripción general y especificación del problema, incluyendo los casos de prueba. Esto lo decidimos porque sentimos que esos son los puntos más débiles de cada uno, por lo que trabajar en lo que no conocemos tanto nos ayudaría a mejorar, aunque luego descartaríamos esa idea porque no queremos que uno conozca mucho del tema y otro poco ya que esto nos puede pasar una mala jugada en la sustentación del proyecto.

Miércoles 6 de mayo de 2026: Definimos por completo la descripción general del problema, en primera instancia lo primero que notamos es que este programa tiene similitudes con problemas subidos anteriormente en la arena, específicamente los problemas Together (en la tarea 5 del curso de arboles y grafos 2026-1) y die, (en la tarea 3 del curso de estructuras de datos de 2025-2). Las similitudes que encontramos respectivamente son, grafos de estado y la orientación del dado o cubo al rotarlo en las posibles direcciones a las cuales nos podemos mover. Juan José no vio el curso de estructuras de datos con Carlos, pero si lo vio con Gerardo, por lo que, si realizó la tarea y recordaba usar un struct para el dado, por lo que un primer acercamiento al código podría ser ese. Por otro lado, Yura denotó que al ser un grafo de estado lo primero que se le llegó a la cabeza fue hacer un bfs o un Dijkstra a lo cual Juan José estuvo de acuerdo, por lo que como primer acercamiento al código tenemos esos pequeños incisos.

Sábado 9 de mayo de 2026: Realizamos la especificación del problema, decidimos realizarlo en forma de tuplas ya que es la forma mas efectiva y entendible en la que tu puedes

definir un grafo de estados, esto ya lo teníamos entendido ya que habíamos estudiado con anterioridad el problema de sokoban, el cual luego apareció en el parcial, para este teníamos que definir de la misma manera este grafo de estados, por lo que teníamos en mente lo que teníamos que realizar. Juan José no tenía muy claro porque las posiciones del oro se guardaban en un vector de máximo 6 posiciones, él pensaba que se tenía que guardar constantemente el mapa completo para poder visualizar las posiciones del oro, pero Yura le explicó que esto además de ser poco eficiente ya que estaríamos guardando el mapa completo, esto directamente no es correcto, ya que el mapa no se considera como un estado como tal, ya que aunque las posiciones en las que está el oro pueden cambiar, el mapa como tal NO cambia, por lo que no se considera como un estado.

Martes 12 de mayo de 2026: En clase el profe especificó que NO podíamos mencionar dentro de la especificación del problema cosas como tuplas, ya que eso de cierta manera ya infiere una forma de solucionar el problema, y como la especificación es general, nuestro acercamiento puede llegar a un camino específico, el cual no termine cubriendo todos los casos generales, por lo que nos tocó volver a modificar y realizar de nuevo la especificación del problema.

Jueves 14 de mayo de 2026: Yura y Juan Jose deciden hacer un GitHub para subir los documentos e irlos actualizando automáticamente.

Viernes 15 de mayo de 2026: Al hacer los casos de prueba nos dimos cuenta de que necesitamos guardar la posición de las piezas de oro ya que necesitamos saber en donde están en cada instante, de igual manera usar una lista con las posiciones iniciales en el estado puede que no sea una buena opción, es algo que tenemos que pensar de mejor manera. De igual manera es un buen primer

acercamiento al método de estados finales que vamos a usar.

Sábado 16 de mayo: Antes de entregar el documento Yura y Juan José se reunieron por última vez para revisar y terminar de chequear las especificaciones del problema, encontramos unas inconsistencias entre lo que decíamos y luego lo que analizábamos en los casos de prueba. Fuimos muy rigurosos siendo conscientes de que las especificaciones no tienen que basarse en un caso de prueba específico (como el que nosotros planteamos) si no que tienen que basarse en hacer una cobertura correcta de todas las posibilidades del problema, esto porque nada nos asegura que los casos de prueba finales de este programa sean iguales a los que usamos nosotros para hacer nuestro análisis.

Martes 26 de mayo: En esta etapa nos enfocamos principalmente en traducir las ideas obtenidas durante la especificación del problema a estructuras concretas dentro del código. Implementamos las funciones encargadas de representar las rotaciones del cubo en cada una de las posibles direcciones de movimiento, ya que entendimos que la orientación del cubo es una parte fundamental del estado. También realizamos una primera lectura del mapa y de las posiciones relevantes del problema. Posteriormente refinamos este primer acercamiento definiendo formalmente una estructura de estado que incluyera la posición actual, la orientación del cubo, la información asociada a las caras y el estado de las piezas de oro mediante máscaras de bits. A partir de esto construimos el esqueleto de la solución utilizando Dijkstra, ya que los movimientos pueden tener costos diferentes dependiendo de las acciones realizadas sobre el tablero. Aunque todavía falta implementar parte de la lógica relacionada con la recolección del oro y las transiciones completas entre estados. Nos encontramos con un muro ya que realmente no tenemos idea de como hacer un intercambio entre la matriz y los oros del tablero.

Viernes mayo 29 2026: Hoy fue un día clave para el avance de nuestro proyecto, gracias al muro que nos encontramos, decidimos ir a la oficina del profesor Carlos y este nos aclaró temas del proyecto que no teníamos muy claro. Además, nos mostro una optimización importante en la forma en la que representamos el dado, reduciéndonos el costo a la mitad, por lo que llegando a la casa decidimos hacer varios cambios. Después de analizar la solución que habíamos planteado inicialmente, concluimos que guardar información detallada sobre cada cara del cubo y sobre las posiciones de las piezas de oro estaba haciendo el estado innecesariamente complejo. Por esta razón rediseñamos la estructura principal, pasando de una representación completa de las seis caras a una representación más compacta basada únicamente en las tres caras necesarias para determinar cualquier orientación del cubo. También reemplazamos la forma en que almacenábamos las posiciones del oro, utilizando una máscara de 64 bits que nos permitía representar eficientemente la presencia o ausencia de oro en cada casilla del tablero. A partir de esta nueva representación implementamos las operaciones para verificar, recoger y depositar oro, además de actualizar correctamente el estado después de cada movimiento.

Sábado 30 de mayo de 2026: El día de hoy nos dimos cuenta que nuestro programa está pobre con las optimizaciones, esto ya que subimos el ejercicio (el cual ya es correcto) al contest A de la arena y este nos dio timelimit, si el más básico salió así, significa que nos falta, por lo que dedicamos este día a optimizar la representación de los estados y a reducir significativamente el consumo de memoria y el tiempo de ejecución.

Martes 2 de junio de 2026: Analizando el comportamiento del algoritmo nos dimos cuenta de que almacenar estructuras completas para cada estado dentro de Dijkstra generaba una gran sobrecarga, por lo que buscamos una

representación más compacta. A partir de esto precomputamos las 24 orientaciones posibles del cubo y las transiciones entre ellas, aprovechando que cualquier orientación puede describirse únicamente mediante tres caras. También desarrollamos un sistema de codificación que empaqueta en un solo entero la posición del jugador, la orientación del cubo y la información relacionada con las caras que contienen oro. De igual manera reemplazamos varias estructuras basadas en mapas ordenados por tablas hash, lo que permitió acelerar considerablemente las búsquedas y actualizaciones de estados durante la ejecución del algoritmo.

Descripción de los cambios que se hicieron en la solución desde la primera versión hasta la entrega final:

Desde la primera versión de la solución hasta la entrega final se realizaron cambios importantes tanto en la representación del problema como en la eficiencia de la implementación. Inicialmente el enfoque se basaba en modelar explícitamente todas las características del estado mediante estructuras que almacenaban la orientación completa del cubo, información asociada a cada una de sus caras y el estado de las piezas de oro, utilizando además mapas ordenados para gestionar los estados visitados dentro del algoritmo de Dijkstra. A medida que avanzó el desarrollo identificamos que esta representación generaba una gran cantidad de información redundante y aumentaba considerablemente el costo computacional. Por esta razón se rediseñó el estado para almacenar únicamente la información estrictamente necesaria, reemplazando la representación completa de las seis caras por una descripción basada en tres caras que permite reconstruir cualquier orientación del cubo. Posteriormente se introdujo el uso de máscaras de bits para representar tanto las posiciones del oro en el tablero como las caras del cubo que transportan oro, reduciendo el consumo de memoria y simplificando las

operaciones de actualización. Finalmente se realizó una etapa de optimización donde se precomputaron las 24 orientaciones posibles del cubo y las transiciones entre ellas, se codificaron múltiples componentes del estado dentro de enteros compactos y se reemplazaron estructuras de búsqueda más costosas por tablas hash. Estas modificaciones permitieron conservar la misma lógica general basada en un grafo de estados recorrido mediante Dijkstra, pero obteniendo una solución significativamente más eficiente y adecuada para manejar la gran cantidad de estados que pueden generarse durante la ejecución del problema.